

Exam Ticket 1

(1) Theory:

- Machine learning: definition, key examples of tasks (regression, classification), and types of learning (supervised, unsupervised, reinforcement learning).

(2) Calculation/Derivation:

- Derive the ordinary least squares (OLS) method from the maximum likelihood principle, assuming Gaussian errors.

(3) Programming Practice:

- Write Python code to read an input file where each line is a JSON object. Extract several required fields from each object (for example, `user_id`, `item_id`) and construct a `pandas.DataFrame`.
-

Exam Ticket 2

(1) Theory:

- The sample–feature matrix and its role in ML. Regression and classification tasks: common loss functions (MSE, MAE, logloss, etc.). How do loss functions differ from evaluation metrics?

(2) Calculation/Derivation:

- Derive the maximum likelihood estimate of the parameter of a Bernoulli random variable.

(3) Programming Practice:

- Read a CSV dataset in which one of the features is categorical (for example, `color`). Apply one-hot encoding to this feature using `pandas.get_dummies` or `sklearn.preprocessing.OneHotEncoder`.
-

Exam Ticket 3

(1) Theory:

- Working with categorical features: ordered and unordered features, encoding methods (one-hot encoding, count-based encoding, the hashing trick). When is count-based encoding preferable?

(2) Calculation/Derivation:

- Give the analytical formula for the coefficients of linear regression without regularization. Briefly derive it by minimizing $\|Xw - y\|^2$.

(3) Programming Practice:

- Using Python/NumPy, generate a synthetic dataset with a few dozen points, fit a simple linear regression model using OLS (with the analytical formula or `np.linalg.solve`), print the estimated weights, and visualize the result.
-

Exam Ticket 4

(1) Theory:

- Linear models: L1 and L2 regularization and their properties (sparse weights with L1, weight shrinkage with L2). Can a purely linear model capture a nonlinear relationship?

(2) Calculation/Derivation:

- Give the analytical expression for linear regression with L2 regularization (Ridge).

(3) Programming Practice:

- Implement a Python function to compute the ROC curve and ROC AUC from given true labels y_{true} and predicted probabilities $y_{\text{pred_proba}}$. Compare the results with `sklearn.metrics.roc_curve` and `roc_auc_score`.
-

Exam Ticket 5

(1) Theory:

- Binary classification metrics: precision, recall, TPR, FPR, ROC, ROC AUC, and F1 score. How is a classifier's decision threshold selected?

(2) Calculation/Derivation:

- Write the formulas for precision, recall, and F1. Describe how to construct the ROC curve and compute ROC AUC.

(3) Programming Practice:

- Write a function that accepts a list of text strings (or several strings), extracts a vocabulary of unique words, and returns this set as an “alphabet” for further processing, such as text classification.
-

Exam Ticket 6

(1) Theory:

- Decision trees: the construction procedure (greedy splitting) and split quality criteria — entropy, variance, and Gini impurity. How is the prediction at a regression leaf selected?

(2) Calculation/Derivation:

- What does “greedy” mean when choosing a split? Write the formula for Gini impurity and briefly explain its meaning.

(3) Programming Practice:

- Use scikit-learn to train a decision tree, for example on the Iris dataset. Print its depth and feature importances, visualize the tree using `export_graphviz` or `plot_tree`, and show the result.
-

Exam Ticket 7

(1) Theory:

- Bagging and Random Forest: what is the idea behind bagging? How is a random forest constructed? How do hyperparameters (the number of trees, their depth, and `max_features`) affect the bias–variance tradeoff?

(2) Calculation/Derivation:

- The entropy formula is $H(p) = -\sum_i p_i \log p_i$. Under what conditions is entropy minimized or maximized (for k equally possible outcomes)?

(3) Programming Practice:

- Write a Python function that accepts a list of strings, each containing a JSON object, and returns a `pandas.DataFrame` containing only the required fields, discarding everything else.
-

Exam Ticket 8

(1) Theory:

- Gradient-boosted trees: what is the idea behind the “gradient” step? How are trees built iteratively to approximate the negative gradient of the loss function?

(2) Calculation/Derivation:

- The general form of logloss is:

$$\text{logloss} = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right].$$

Show how minimizing logloss yields an estimate of the Bernoulli parameter equal to the observed average frequency of success.

(3) Programming Practice:

- Train a gradient boosting model, such as `XGBClassifier` or `LightGBM`, on real data. Tune the main hyperparameters (`learning_rate`, `n_estimators`, etc.) and evaluate performance using ROC AUC.
-

Exam Ticket 9

(1) Theory:

- SVD (singular value decomposition): the main idea, its relationship to matrix factorization, and applications in recommender systems, image compression, etc.

(2) Calculation/Derivation:

- Write the expression for $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$. Explain what a rank-1 matrix is and how it can be expressed in terms of the columns of $\mathbf{U}$ and $\mathbf{V}$ and the diagonal entries of $\mathbf{S}$.

(3) Programming Practice:

- Generate a random matrix, for example with `np.random.randn(10, 5)`, perform SVD using `np.linalg.svd`, and store $\mathbf{U}$, $\mathbf{S}$, $\mathbf{V}^\top$. Verify that $\mathbf{U} @ \text{np.diag}(\mathbf{S}) @ \mathbf{V}^\top$ matches the original matrix within numerical precision.
-

Exam Ticket 10

(1) Theory:

- Confidence intervals and hypothesis testing: what are the confidence level, statistical power, null hypothesis, and statistical significance?

(2) Calculation/Derivation:

- A conditional probability problem about taxis: in a city, 15 % of taxis are “Blue” and 85 % are “Green”. At night, a witness identifies the color correctly with probability 80 %. The witness says the taxi was “Blue”. What is the probability that the taxi was actually “Blue”?

(3) Programming Practice:

- Generate samples from normal distributions with different means and sample sizes. Construct a confidence interval for the mean of each sample using, for example, `statsmodels` or `scipy.stats`, and compare the results across sample sizes.
-

Exam Ticket 11

(1) Theory:

- Embeddings: what are they and why are they useful? Example: word2vec — how does it generalize the idea of representing words as vectors?

(2) Calculation/Derivation:

- A conditional probability problem about a virus: the XYZ virus affects 1 in 1000 people. A test has a 5 % false positive rate. A person tests positive. What is the probability that the person is actually infected?

(3) Programming Practice:

- Implement a simple trainable embedding matrix in NumPy or PyTorch/TensorFlow for a set of unique words. Demonstrate how it works using a toy example.
-

Exam Ticket 12

(1) Theory:

- The Bayesian approach versus the classical (frequentist) approach: estimation of Gaussian and Bernoulli parameters, and differences in the interpretation of confidence intervals. Bayes' theorem.

(2) Calculation/Derivation:

- Derive Bayes' theorem from the definition of conditional probability and explain its main components: the prior, likelihood, and normalization constant.

(3) Programming Practice:

- Implement a naive Bayes classifier (BernoulliNB) for binary classification using NumPy or scikit-learn. Compare it with `sklearn.naive_bayes.BernoulliNB`.
-

Exam Ticket 13

(1) Theory:

- The naive Bayes classifier and conjugate distributions for the Gaussian and Bernoulli cases. Why is conjugacy useful in Bayesian models?

(2) Calculation/Derivation:

- Derive the derivative (gradient) of the sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$.

(3) Programming Practice:

- Implement backpropagation for logistic regression in NumPy: derive the gradient with respect to the weights $\mathbf{w}$ using the logloss function.
-

Exam Ticket 14

(1) Theory:

- An overview of methods: KNN, naive Bayes, linear models, SVM, decision trees, and ensembles (bagging, boosting, stacking). How are they related to neural networks, and how do they differ?

(2) Calculation/Derivation:

- Differentiate $\log(\det(\mathbf{A}))$ with respect to the matrix $\mathbf{A}$.

(3) Programming Practice:

- Write code to generate a synthetic two-dimensional dataset with two classes. Train `sklearn.svm.SVC` and visualize its decision boundary together with the original data points.
-

Exam Ticket 15

(1) Theory:

- Neural networks: what are backpropagation and matrix differentiation? Different activation functions (sigmoid, tanh, ReLU, etc.) and their properties (monotonicity, saturation, differentiability).

(2) Calculation/Derivation:

- Find the derivative of $\tanh(x)$ with respect to its argument. How does $\tanh(x)$ differ from $\sigma(x)$ in terms of its range?

(3) Programming Practice:

- Implement a single-layer perceptron in NumPy, including forward and backward passes. Use a small synthetic dataset to verify that the training error decreases.
-

Exam Ticket 16

(1) Theory:

- RNNs: their architecture, constituent matrices and vectors, state updates at each step, and the idea of training an RNN using backpropagation through time.

(2) Calculation/Derivation:

- The derivative of the RNN loss function with respect to the logits at each step. Describe the main steps of backpropagation through time: how does the gradient propagate across time steps?

(3) Programming Practice:

- Implement a simplified RNN for next-character prediction (character-level generation). Generate a small training set, train the model, and try generating text.
-

Exam Ticket 17

(1) Theory:

- LSTM: how does it differ from a classic RNN? The idea of “long short-term memory” (gates) and its advantages over a standard RNN. Briefly mention the universal approximation theorem.

(2) Calculation/Derivation:

- How is Adagrad implemented in NumPy? What role does exponential smoothing play in optimizers such as RMSProp and Adam? Give the formula for an exponential moving average.

(3) Programming Practice:

- Implement RNN text generation in Python with a “temperature” parameter. Explain how temperature affects the “creativity” of the generated text.
-